

UNIVERSIDAD ABIERTA INTERAMERICANA



FACULTAD DE TECNOLOGÍA INFORMÁTICA

Uso de IA DAW

Materia: Desarrollo y Arquitecturas Web

Docente: Santiago Nahuel Zavatti, Veronica Sofia Casado

Facultad: Universidad Abierta Interamericana

Sede: Roca

Curso: 4to año – Turno Tarde

Alumno: Bruno Pignani

Año: 2026

Herramienta utilizada: Gemini

Modelo utilizado: Gemini 3.1 Pro + Razonamiento ampliado

Primera consulta

Prompt: "Necesito desarrollar el motor lógico para un juego de Blackjack estructurado en JavaScript ES5 estricto. ¿Cómo puedo inicializar un array de objetos para representar la baraja francesa y qué algoritmo matemático recomiendas para mezclarlo aleatoriamente?"

Respuesta: "Para mantener el cumplimiento estricto con ES5, declararemos todas las variables al inicio de las funciones utilizando var. Para inicializar la baraja, la mejor aproximación es utilizar dos bucles for anidados que combinen los palos y los valores en un array de objetos.

En cuanto a la mezcla aleatoria, el estándar de la industria es el **Algoritmo de Fisher-Yates**. Este algoritmo recorre el array desde el final hacia el principio, intercambiando cada elemento con otro elegido al azar mediante `Math.random()`, lo que garantiza una distribución probabilística perfecta.

Aquí tienes el código de implementación:"

```
"use strict";

var mazo = [];
var palos = ["♠", "♣", "♥", "♦"];
var valores = ["A", "2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K"];

// Inicialización de la baraja
function crearMazo() {
    var i, j, carta;
    mazo = [];
    for (i = 0; i < palos.length; i++) {
        for (j = 0; j < valores.length; j++) {
            carta = {
                palo: palos[i],
                valor: valores[j],
                puntaje: calcularPuntaje(valores[j]) // Función auxiliar para el valor numérico
            };
            mazo.push(carta);
        }
    }
}

// Algoritmo de Fisher-Yates para mezcla aleatoria
function mezclarMazo() {
    var i, j, cartaTemporal;
    for (i = mazo.length - 1; i > 0; i--) {
        j = Math.floor(Math.random() * (i + 1));
        cartaTemporal = mazo[i];
        mazo[i] = mazo[j];
    }
}
```

```

    mazo[j] = cartaTemporal;
  }
}

```

Explicación del uso: Se utilizó para estructurar la base matemática del proyecto (aislada en motor.js). La asistencia permitió establecer un modelo de datos robusto para la inicialización de las cartas y garantizar una aleatoriedad real en el reparto aplicando un algoritmo estandarizado de la industria, cumpliendo con la restricción de no depender de librerías externas.

Segunda consulta

Prompt: "Ya tengo la lógica de la baraja aislada. Necesito que, al interactuar con el botón de apostar, el sistema inyecte dinámicamente las cartas en el HTML sin recargar el sitio. ¿Cuál es la forma más limpia de hacerlo respetando la separación de responsabilidades?"

Respuesta: "Para respetar la separación de responsabilidades y evitar inyecciones directas de strings HTML (lo cual es una mala práctica), la forma más limpia es utilizar los métodos nativos del DOM: document.createElement() para generar el nodo y appendChild() para insertarlo en la vista.

De esta manera, tu motor lógico (motor.js) solo pasará el objeto de la carta, y el archivo de la interfaz (juego.js) se encargará de renderizarla dinámicamente.

```

function dibujarCartaEnMesa(carta, contenedorDestino) {
  // 1. Se crea el elemento <div> dinámicamente
  var nuevaCartaVisual = document.createElement("div");

  // 2. Se le asigna la clase CSS para el diseño
  nuevaCartaVisual.classList.add("carta-fisica");

  // 3. Se inyecta el contenido de texto (ej: "10♠")
  nuevaCartaVisual.textContent = carta.valor + carta.palo;

  // 4. Lógica visual: Se aplica color rojo si es Corazones o Diamantes
  if (carta.palo === "♥" || carta.palo === "♦") {
    nuevaCartaVisual.style.color = "red";
  }

  // 5. Se inserta el nodo finalizado en el contenedor del HTML
  contenedorDestino.appendChild(nuevaCartaVisual);
}

```

Explicación del uso: Se empleó para conectar de forma segura el motor matemático con la interfaz gráfica (juego.js). La herramienta ayudó a implementar una actualización dinámica del DOM utilizando métodos nativos, evitando la inyección directa de cadenas de texto

(strings HTML) para mantener el cumplimiento estricto de las buenas prácticas de arquitectura frontend.

Tercera consulta

Prompt: “Según la consigna enviada verifica que se cumplan con todos los requerimientos y reglas que pide el trabajo práctico y luego enuncia todo lo que se debería corregir”

Respuesta: Acá tenés el historial de los errores que tenía el proyecto antes de pulirlo:

● 1. Violaciones en HTML

- **Estilos en línea:** Tenías atributos `style="..."` inyectados directamente en las etiquetas de `index.html` y `contacto.html` (por ejemplo, en los botones, enlaces y en el textarea), lo cual rompía la regla estricta de "No usar estilos inline" [cite: 15].
- **Mal uso de etiquetas de salto:** En la pantalla de "Cómo Jugar", estabas utilizando la etiqueta `<br>` para apilar visualmente las instrucciones del teclado, violando la norma "No usar `<br>` para maquetar" [cite: 15].
- **Falta de vinculación del Reset:** El archivo `contacto.html` no tenía la etiqueta `<link>` apuntando al reseteo de estilos, incumpliendo la directiva de "Utilizar `reset.css`" de manera uniforme en el sitio [cite: 15].

● 2. Violaciones en JavaScript (ES5)

- **Declaración dispersa de variables:** En los archivos `motor.js` y `juego.js`, declarabas variables (usando `var`) a mitad de las funciones o directamente adentro de los bucles `for` (ej: `var i = 0`). La rúbrica exigía explícitamente "Declarar variables al inicio" de cada función para cumplir con las buenas prácticas antiguas de ES5 [cite: 15].
- **Código duplicado (Falta de prolijidad):** En tu archivo `juego.js`, el evento del botón "Comenzar" tenía un bloque de código repetido dos veces por un error de copiado y pegado, lo cual te iba a restar puntos en el criterio de entregar un "Código prolijo y estricto" [cite: 15].

● 3. Violaciones en CSS

- **Falta de Responsividad Móvil (Media Queries):** Tu hoja de estilos original (`styles.css`) no tenía código específico para celulares, lo que fallaba en cumplir con la "Responsividad y estética" exigida, y la estructura donde "Los media queries deben ir al final" [cite: 15].
- **Reset vacío:** Al principio, tu archivo `reset.css` estaba completamente vacío, por lo que no cumplía ninguna función real a pesar de estar vinculado [cite: 15].

Explicación del uso: Se utilizó la herramienta como auditor de código estático cruzando el desarrollo contra las reglas de la consigna. El objetivo fue garantizar que el diseño final respetara la separación de capas (HTML semántico sin estilos en línea), la correcta declaración de variables exigida por el estándar ES5 y la implementación efectiva de Media Queries.

Cuarta consulta

Prompt: “Quiero que me des un recuento de bugs que fuimos fixeados a medida que fui desarrollando el juego”

Respuesta: Es verdad, el proyecto tuvo una evolución enorme desde el primer esqueleto hasta llegar a la versión final de hoy. Esos errores de las primeras fases son súper comunes en el desarrollo y arreglarlos fue lo que le dio tanta solidez a la arquitectura de tu juego. Acá tenés el recuento de esos "bugs" de desarrollo que fuimos cazando y resolviendo progresivamente durante las pruebas:

1. Problemas de Interfaz y DOM

- **Superposición de pantallas (Login y Mesa):** Al principio, cuando cargabas la página, se veía el formulario de ingreso mezclado con la mesa verde de Blackjack. Lo solucionamos implementando la clase CSS `.oculto (display: none !important;)` y manipulando el DOM con `classList.add()` y `classList.remove()` para hacer una transición limpia.
- **Pérdida de foco en los inputs:** Cuando pasabas del login a la mesa, tenías que hacer clic manualmente en la caja de apuestas. Lo arreglamos agregando un `setTimeout` de 100 milisegundos que le da tiempo al navegador para renderizar la pantalla antes de ejecutar `inputApuesta.focus()`.

2. Problemas de Lógica y Botones

- **Botones "congelados":** Durante el testeo nos pasó que, al terminar una mano, los botones de "Pedir", "Plantarse" o "Apostar" quedaban bloqueados (`disabled = true`) de forma permanente, obligando a recargar la página. Lo arreglamos en la función `determinarGanador()`, asegurándonos de devolverles el estado `disabled = false` al finalizar cada ronda.
- **Manejo del scroll con el teclado:** Cuando implementamos las flechas direccionales para jugar, apretar la flecha derecha o izquierda movía toda la página web hacia los costados. Lo reparamos capturando el evento del teclado e inyectando `evento.preventDefault()` para anular el comportamiento nativo del navegador.
- **Interrupción de escritura:** Al usar la tecla "Enter" como atajo para apostar o repartir, si el jugador intentaba escribir su nombre y apretaba Enter, se disparaban las funciones del juego por error. Le pusimos una "traba de seguridad" detectando si el usuario estaba escribiendo con `document.activeElement.tagName === "INPUT"`.

3. Problemas del Motor Matemático

- **El historial volátil:** El historial visual funcionaba perfecto mientras la pestaña estaba abierta, pero al cerrarla se borraba (a diferencia del saldo, que sí quedaba). Lo parcheamos guardando la estructura visual entera de la lista (`listaHistorial.innerHTML`) adentro del `LocalStorage` bajo el nombre de cada jugador.

Explicación del uso: Se empleó para consolidar un registro histórico de la evolución del proyecto y documentar la resolución de problemas. Esto permitió estructurar claramente los desafíos técnicos superados durante las etapas de pruebas, tales como el asincronismo del DOM, el manejo de eventos preventivos del teclado y la gestión del estado de la interfaz gráfica.

Quinta consulta

Prompt: "Para cumplir con el requerimiento de persistencia de datos, necesito guardar el saldo del jugador y su historial de partidas en el navegador. ¿Cómo puedo implementar

LocalStorage en ES5 asegurando que, si el usuario es nuevo, se inicialice un saldo por defecto, y si ya existe, recupere su estado anterior?"

Respuesta: "Para manejar la persistencia de estado en el navegador utilizando ES5, el objeto nativo `localStorage` es la herramienta ideal. Debemos estructurar una lógica de control (un condicional `if/else`) al momento de que el usuario ingrese su nombre. Utilizaremos `localStorage.getItem()` concatenando una clave única con el nombre del jugador para evitar que los datos se mezclen si juegan distintas personas en la misma computadora.

Aquí tienes la implementación para la carga inicial y la función para actualizar los datos en tiempo real:"

```
// Lógica de recuperación al iniciar sesión
var saldoGuardado = localStorage.getItem("Blackjack_Saldo_" + nombreJugador);
var historialGuardado = localStorage.getItem("Blackjack_Historial_" + nombreJugador);

if (saldoGuardado !== null) {
    // Si existe, parseamos el string a entero
    saldoJugador = parseInt(saldoGuardado);
} else {
    // Si es un jugador nuevo, inicializamos el valor por defecto
    saldoJugador = 500;
}

if (historialGuardado !== null) {
    // Restauramos el historial visual
    listaHistorial.innerHTML = historialGuardado;
} else {
    // Vaciamos la lista para un jugador nuevo
    listaHistorial.innerHTML = "";
    numeroMano = 0;
}

// Función aislada para actualizar el almacenamiento al finalizar cada mano
function actualizarYGuardarSaldo() {
    spanSaldo.textContent = saldoJugador;
    // Guardamos el nuevo valor en el navegador
    localStorage.setItem("Blackjack_Saldo_" + nombreJugador, saldoJugador);
}
```

Explicación del uso: La asistencia permitió estructurar correctamente las claves dinámicas en el `LocalStorage` (asociadas al nombre de usuario) y manejar el flujo de inicialización del estado.